

Machine Learning Fundamentals to Practice

COMPREHENSIVE LECTURE HANDOUT

Concepts • Algorithms • Evaluation • Applications

Contents

1	Introduction to Machine Learning	3
1.1	Where ML Fits: AI Landscape	3
1.2	Why Machine Learning?	3
2	Types of Machine Learning	4
2.1	Supervised Learning	4
2.2	Unsupervised Learning	4
2.3	Reinforcement Learning	4
2.4	Semi-Supervised and Self-Supervised Learning	5
3	Key Terminology and Concepts	5
4	The Machine Learning Pipeline	6
4.1	Step 1: Data Collection	6
4.2	Step 2: Data Cleaning and Preprocessing	6
4.3	Step 3: Feature Engineering	6
4.4	Step 4: Train/Test Split	6
4.5	Step 5–7: Training, Evaluation, Tuning	6
5	Supervised Learning Algorithms	7
5.1	Linear Regression	7
5.2	Logistic Regression	7
5.3	Decision Trees	8
5.4	K-Nearest Neighbors (KNN)	8
5.5	Support Vector Machines (SVM)	8
5.6	Naïve Bayes	9
6	Ensemble Methods	9
6.1	Bagging (Bootstrap Aggregating)	10
6.2	Boosting	10
6.3	Bagging vs. Boosting	10

7	Unsupervised Learning Algorithms	11
7.1	K-Means Clustering	11
7.2	Hierarchical Clustering	11
7.3	Principal Component Analysis (PCA)	11
8	Neural Networks and Deep Learning	12
8.1	Key Components	12
8.2	Common Deep Learning Architectures	12
9	Model Evaluation and Metrics	13
9.1	Bias–Variance Tradeoff	13
9.2	Overfitting vs. Underfitting	13
9.3	Classification Metrics	14
9.4	Regression Metrics	14
9.5	Cross-Validation	14
10	Regularization Techniques	14
11	Practical Considerations	15
11.1	Handling Imbalanced Data	15
11.2	Hyperparameter Tuning	15
11.3	Feature Importance and Interpretability	16
12	Summary: Algorithm Selection Guide	16
13	Best Practices Checklist	16
14	Common Pitfalls	17
15	ML Workflow — End-to-End Example	17
16	Review Questions	18

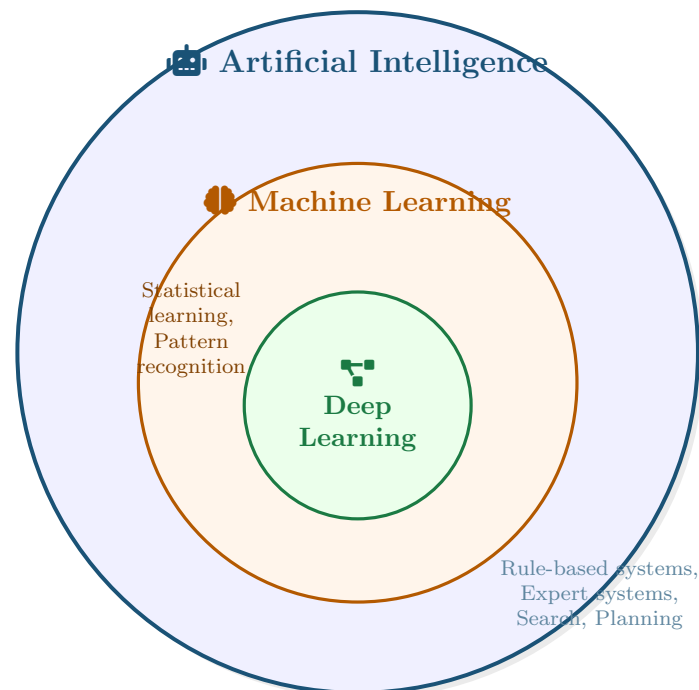
1 Introduction to Machine Learning

Machine Learning

A subfield of Artificial Intelligence (AI) that gives computers the ability to **learn from data** and make predictions or decisions **without being explicitly programmed** for every scenario.

“A computer program is said to learn from experience E with respect to some task T and some performance measure P , if its performance on T , as measured by P , improves with experience E .” — Tom Mitchell (1997)

1.1 Where ML Fits: AI Landscape

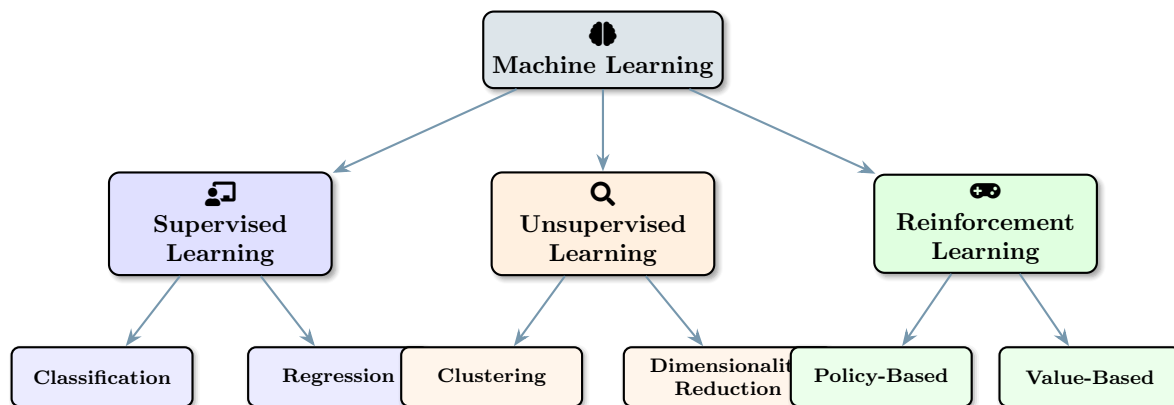


- **AI** is the broadest field—any technique enabling machines to mimic human intelligence.
- **Machine Learning** is a subset of AI that learns patterns from data automatically.
- **Deep Learning** is a subset of ML using neural networks with many layers.

1.2 Why Machine Learning?

- Problems too complex to program explicitly (e.g., image recognition, language translation).
- Automatically adapts to new data without manual reprogramming.
- Discovers hidden patterns in large datasets that humans cannot easily find.
- Powers modern applications: search engines, recommendations, self-driving cars, medical diagnosis.

2 Types of Machine Learning



2.1 Supervised Learning

Supervised Learning

The algorithm learns from **labeled training data**—input-output pairs (x_i, y_i) —to learn a mapping function $f : X \rightarrow Y$ that can predict outputs for new, unseen inputs.

- **Classification:** Predicting a *discrete* class label.
Examples: Spam detection (spam/not spam), disease diagnosis (positive/negative), image recognition (cat/dog).
- **Regression:** Predicting a *continuous* numerical value.
Examples: House price prediction, stock price forecasting, temperature prediction.

2.2 Unsupervised Learning

Unsupervised Learning

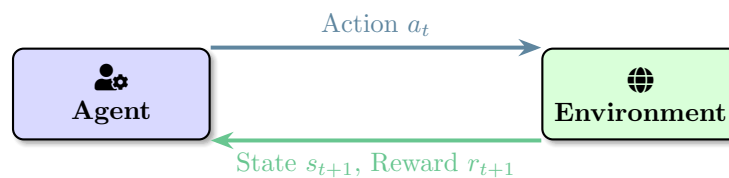
The algorithm learns from **unlabeled data**—finding hidden patterns, structures, or groupings in the data without predefined outputs.

- **Clustering:** Grouping similar data points together.
Examples: Customer segmentation, document grouping, anomaly detection.
- **Dimensionality Reduction:** Reducing the number of features while preserving important information.
Examples: PCA, t-SNE, autoencoders—used for visualization and noise reduction.
- **Association Rule Learning:** Finding interesting relationships between variables.
Examples: Market basket analysis (“customers who bought X also bought Y”).

2.3 Reinforcement Learning

Reinforcement Learning

An agent learns to make decisions by performing **actions** in an **environment** to maximize cumulative **reward**. It learns through trial and error.



Examples: Game playing (AlphaGo, Atari), robotics, autonomous driving, recommendation systems.

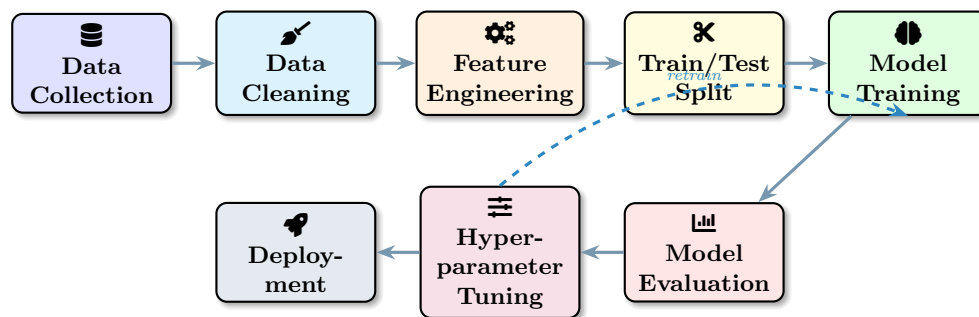
2.4 Semi-Supervised and Self-Supervised Learning

- **Semi-Supervised:** Uses a small amount of labeled data combined with a large amount of unlabeled data.
- **Self-Supervised:** The model generates its own labels from the data (e.g., predicting the next word in a sentence). Foundation for large language models (GPT, BERT).

3 Key Terminology and Concepts

Term	Definition
Feature (Variable)	An individual measurable property of the data (input column), e.g., age, income, pixel value.
Label (Target)	The output variable the model is trying to predict, e.g., spam/not spam, price.
Training Set	The portion of data used to train (fit) the model.
Validation Set	Data used to tune hyperparameters and prevent overfitting during training.
Test Set	Held-out data used to evaluate the final model's performance on unseen data.
Model	The mathematical function learned from data that maps inputs to outputs.
Hyperparameter	A parameter set <i>before</i> training (e.g., learning rate, number of trees, k in KNN).
Parameter	A value learned <i>during</i> training (e.g., weights in linear regression, neural network weights).
Epoch	One complete pass through the entire training dataset.
Batch Size	Number of training samples used in one iteration of model update.
Loss Function	A function that measures how far the model's predictions are from actual values.
Gradient Descent	An optimization algorithm that iteratively adjusts parameters to minimize the loss function.

4 The Machine Learning Pipeline



4.1 Step 1: Data Collection

Gather data from databases, APIs, web scraping, sensors, surveys, or public datasets. Quality and quantity of data directly impact model performance.

4.2 Step 2: Data Cleaning and Preprocessing

- **Handle missing values:** Imputation (mean, median, mode), removal, or advanced methods (KNN imputation).
- **Remove duplicates** and correct inconsistent entries.
- **Handle outliers:** IQR method, Z-score, domain knowledge.
- **Data type conversions:** Ensure proper types (numeric, categorical, datetime).

4.3 Step 3: Feature Engineering

Feature Engineering

The process of transforming raw data into meaningful features that better represent the underlying problem, improving model accuracy.

- **Encoding categorical variables:** One-hot encoding, label encoding, target encoding.
- **Feature scaling:** Standardization ($z = \frac{x-\mu}{\sigma}$) or Min-Max normalization ($x' = \frac{x-x_{\min}}{x_{\max}-x_{\min}}$).
- **Feature creation:** Polynomial features, interaction terms, domain-specific features.
- **Feature selection:** Removing irrelevant or redundant features (correlation analysis, mutual information, recursive feature elimination).

4.4 Step 4: Train/Test Split

Data Splitting

Typically split data into **training set** (70–80%) and **test set** (20–30%). For hyperparameter tuning, further split training data into training and **validation set**, or use **cross-validation**.

4.5 Step 5–7: Training, Evaluation, Tuning

These are detailed in the following sections.

5 Supervised Learning Algorithms

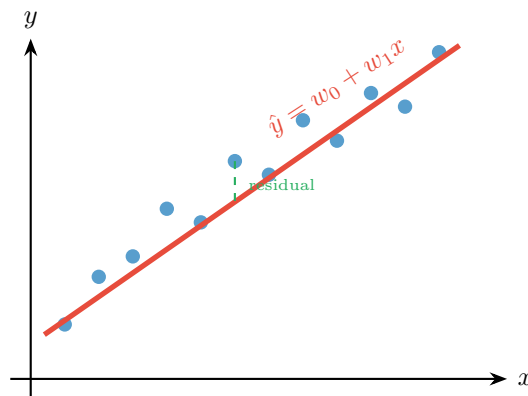
5.1 Linear Regression

Linear Regression

Models the relationship between a dependent variable y and one or more independent variables x by fitting a linear equation:

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + \cdots + w_nx_n = \mathbf{w}^T \mathbf{x} + b$$

Minimizes the **Mean Squared Error (MSE)**: $\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$



- **Assumptions:** Linearity, independence, homoscedasticity, normality of residuals.
- **Regularization:** Ridge (L2: $+\lambda \sum w_i^2$), Lasso (L1: $+\lambda \sum |w_i|$), Elastic Net (L1 + L2).

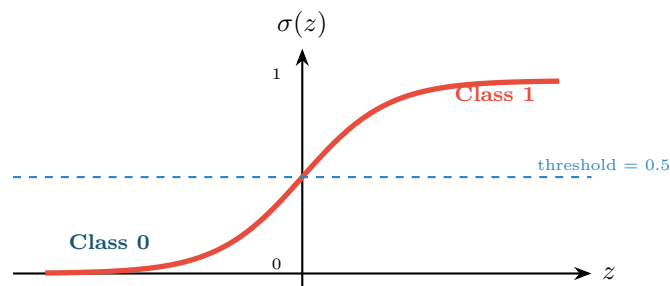
5.2 Logistic Regression

Logistic Regression

A classification algorithm that models the probability of a binary outcome using the **sigmoid function**:

$$P(y = 1 | x) = \sigma(\mathbf{w}^T \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}}$$

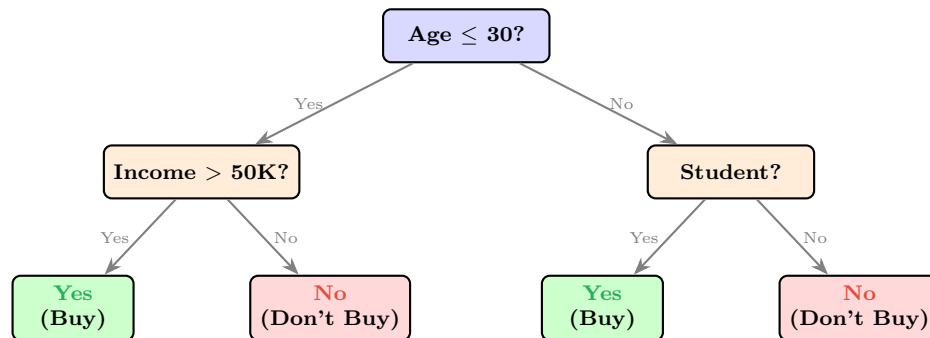
Output is a probability between 0 and 1; a threshold (typically 0.5) determines the class.



5.3 Decision Trees

Decision Tree

A tree-structured model that makes decisions by recursively splitting data based on feature values. Each internal node represents a test on a feature, each branch represents an outcome, and each leaf node represents a class label or value.



- **Splitting criteria:** Gini Impurity, Information Gain (Entropy), Variance Reduction.
- **Gini Impurity:** $G = 1 - \sum_{k=1}^K p_k^2$ (lower is purer)
- **Entropy:** $H = - \sum_{k=1}^K p_k \log_2 p_k$
- **Pros:** Easy to interpret, handles both numerical and categorical data, no feature scaling needed.
- **Cons:** Prone to overfitting, unstable (small data changes → different trees).

5.4 K-Nearest Neighbors (KNN)

KNN

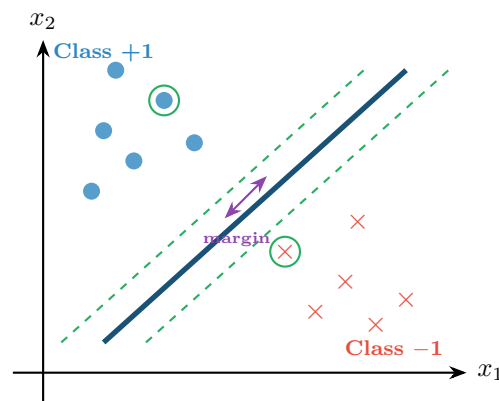
A **non-parametric, instance-based** algorithm that classifies a new data point based on the majority class of its k nearest neighbors in the feature space.

- **Distance metrics:** Euclidean ($\sqrt{\sum (x_i - y_i)^2}$), Manhattan ($\sum |x_i - y_i|$), Minkowski.
- **Choosing k :** Small k = high variance (overfitting); large k = high bias (underfitting). Use cross-validation.
- **Pros:** Simple, no training phase, works with multi-class.
- **Cons:** Slow on large datasets, sensitive to feature scaling, curse of dimensionality.

5.5 Support Vector Machines (SVM)

SVM

Finds the optimal **hyperplane** that maximizes the **margin** (distance) between two classes. Support vectors are the data points closest to the hyperplane.



- **Kernel Trick:** Maps data to higher dimensions for non-linear separation. Common kernels: Linear, Polynomial, RBF (Gaussian).
- **Pros:** Effective in high dimensions, robust to overfitting (with proper regularization).
- **Cons:** Slow on very large datasets, requires feature scaling, not easily interpretable.

5.6 Naïve Bayes

📖 Naïve Bayes

A probabilistic classifier based on **Bayes' Theorem** with a “naïve” assumption that all features are **conditionally independent** given the class:

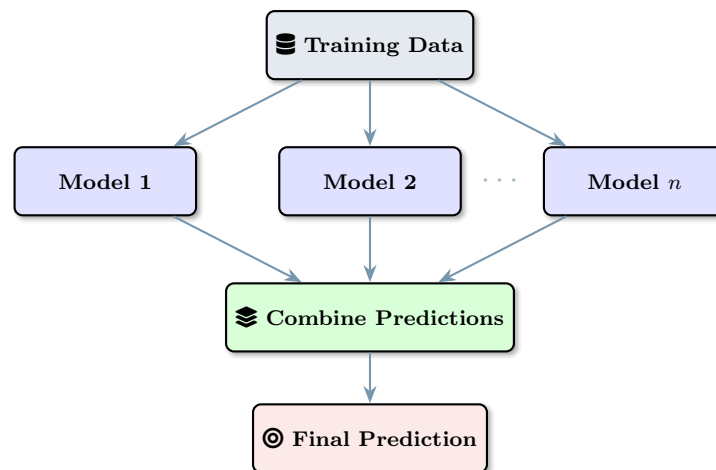
$$P(C | X) = \frac{P(X | C) \cdot P(C)}{P(X)} \propto P(C) \prod_{i=1}^n P(x_i | C)$$

- **Variants:** Gaussian (continuous data), Multinomial (word counts), Bernoulli (binary features).
- **Pros:** Very fast, works well with high-dimensional data, excellent for text classification.
- **Cons:** Independence assumption rarely holds, poor probability estimates.

6 Ensemble Methods

💡 Ensemble Learning

Combining multiple models to produce a stronger model than any individual one. The key idea: diverse models make different errors, and combining them reduces overall error.



6.1 Bagging (Bootstrap Aggregating)

- Train multiple models on different **bootstrap samples** (random subsets with replacement).
- Combine via **majority voting** (classification) or **averaging** (regression).
- **Reduces variance**, helps prevent overfitting.
- **Key example: Random Forest.**

Random Forest

An ensemble of decision trees trained on bootstrap samples with **random feature subsets** at each split. Highly accurate, handles high-dimensional data, provides feature importance scores.

6.2 Boosting

- Train models **sequentially**—each new model focuses on the errors of the previous one.
- **Reduces bias and variance.**
- **Key algorithms:** AdaBoost, Gradient Boosting, XGBoost, LightGBM, CatBoost.

Gradient Boosting

Builds trees sequentially where each tree fits the **residual errors** (negative gradient of the loss function) of the previous ensemble. XGBoost, LightGBM, and CatBoost are highly optimized implementations widely used in competitions and industry.

6.3 Bagging vs. Boosting

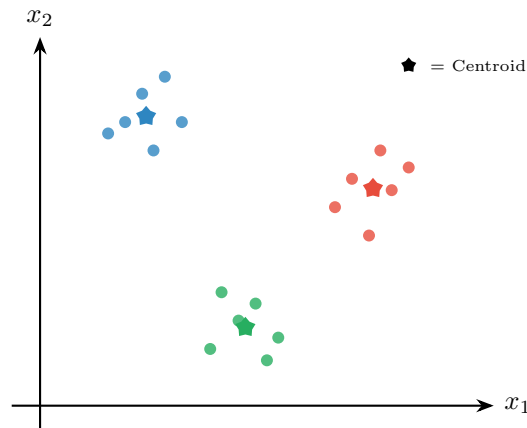
Bagging (e.g., Random Forest)	Boosting (e.g., XGBoost)
Models trained in parallel	Models trained sequentially
Each model trained on bootstrap sample	Each model corrects predecessors' errors
Reduces variance	Reduces bias (and variance)
Less prone to overfitting	Can overfit if not regularized
Equal weight for all models	Models weighted by performance

7 Unsupervised Learning Algorithms

7.1 K-Means Clustering

K-Means

Partitions n data points into k clusters by iteratively assigning points to the nearest **centroid** and updating centroids as the mean of assigned points. Minimizes **within-cluster sum of squares (WCSS)**.



Algorithm steps:

1. Choose k (number of clusters).
 2. Randomly initialize k centroids.
 3. Assign each point to the nearest centroid.
 4. Recompute centroids as mean of assigned points.
 5. Repeat steps 3–4 until convergence.
- **Choosing k :** Elbow method (plot WCSS vs. k), Silhouette score.
 - **Pros:** Simple, fast, scalable. **Cons:** Assumes spherical clusters, sensitive to initialization and outliers.

7.2 Hierarchical Clustering

- **Agglomerative (bottom-up):** Start with each point as its own cluster; merge closest pairs iteratively.
- **Divisive (top-down):** Start with one cluster; recursively split.
- Visualized using a **dendrogram**; cut at desired level to get k clusters.
- **Linkage methods:** Single, complete, average, Ward's.

7.3 Principal Component Analysis (PCA)

PCA

A dimensionality reduction technique that transforms data into a new coordinate system of **principal components**—orthogonal directions that capture the maximum variance.

$$PC_1 = w_1x_1 + w_2x_2 + \dots + w_nx_n$$

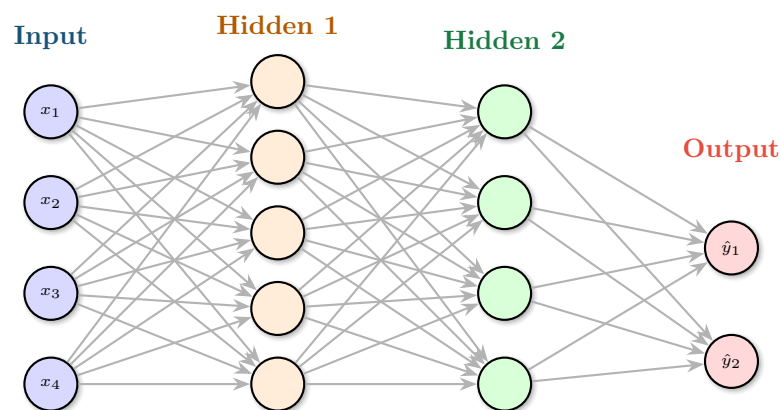
The first principal component explains the most variance, the second the next most, etc.

- **Uses:** Visualization (reduce to 2D/3D), noise reduction, speeding up other algorithms, handling multicollinearity.
- **Steps:** Standardize data → Compute covariance matrix → Compute eigenvectors/eigenvalues → Select top k components.

8 Neural Networks and Deep Learning

Artificial Neural Network (ANN)

A computational model inspired by the human brain, consisting of layers of interconnected **neurons** (nodes). Each connection has a **weight**, and each neuron applies an **activation function** to its weighted input sum.



8.1 Key Components

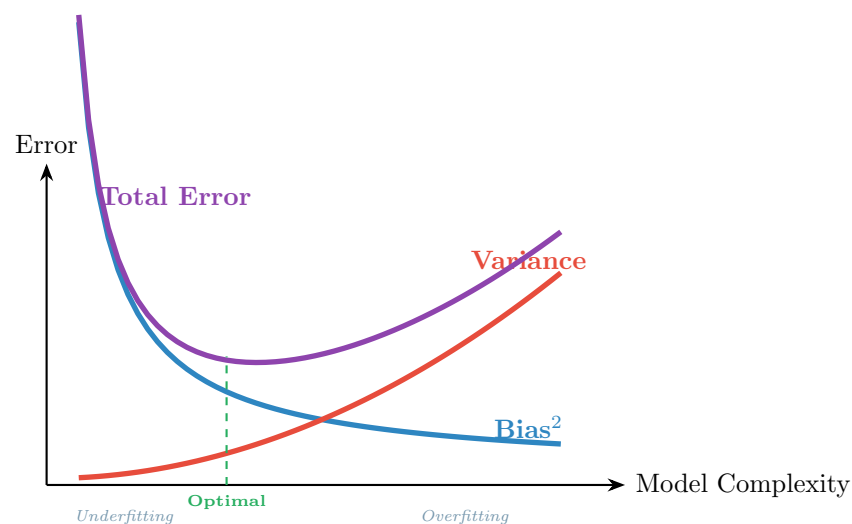
- **Activation Functions:** ReLU ($\max(0, x)$), Sigmoid ($\frac{1}{1+e^{-x}}$), Tanh, Softmax (multi-class output).
- **Forward Propagation:** Input flows through layers; each neuron computes $z = \mathbf{w}^T \mathbf{x} + b$, then $a = \sigma(z)$.
- **Backpropagation:** Computes gradients of the loss w.r.t. weights using the chain rule; updates weights via gradient descent.
- **Optimizers:** SGD, Adam, RMSProp, Adagrad.

8.2 Common Deep Learning Architectures

Architecture	Use Case	Key Idea
CNN (Convolutional Neural Network)	Image classification, object detection	Convolutional filters learn spatial features
RNN (Recurrent Neural Network)	Sequence data, time series	Hidden state captures temporal dependencies
LSTM / GRU	Long sequences, NLP	Gates control information flow, solve vanishing gradient
Transformer	NLP, vision, generative AI	Self-attention mechanism, parallelizable
GAN (Generative Adversarial Network)	Image generation, data augmentation	Generator vs. Discriminator adversarial training
Autoencoder	Dimensionality reduction, anomaly detection	Encoder-decoder learns compressed representation

9 Model Evaluation and Metrics

9.1 Bias–Variance Tradeoff



- **Bias:** Error from oversimplified model (underfitting)—misses relevant relations.
- **Variance:** Error from model being too sensitive to training data (overfitting)—captures noise.
- **Goal:** Find the sweet spot that minimizes *total error* ($\text{bias}^2 + \text{variance} + \text{irreducible error}$).

9.2 Overfitting vs. Underfitting

Underfitting (High Bias)	Overfitting (High Variance)
Model too simple	Model too complex
High training error, high test error	Low training error, high test error
Solution: more features, complex model, less regularization	Solution: more data, regularization, simpler model, dropout

9.3 Classification Metrics

☰ Confusion Matrix

A table summarizing classification results:

		Predicted	
		Positive	Negative
Actual	Positive	TP	FN
	Negative	FP	TN

- **Accuracy:** $\frac{TP+TN}{TP+TN+FP+FN}$ — Overall correctness (misleading on imbalanced data).
- **Precision:** $\frac{TP}{TP+FP}$ — Of predicted positives, how many are actually positive?
- **Recall (Sensitivity):** $\frac{TP}{TP+FN}$ — Of actual positives, how many were correctly identified?
- **F1-Score:** $2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$ — Harmonic mean of precision and recall.
- **ROC Curve:** Plot of TPR vs. FPR at various thresholds. **AUC** (Area Under Curve) summarizes performance.

9.4 Regression Metrics

- **MAE** (Mean Absolute Error): $\frac{1}{n} \sum |y_i - \hat{y}_i|$
- **MSE** (Mean Squared Error): $\frac{1}{n} \sum (y_i - \hat{y}_i)^2$
- **RMSE** (Root MSE): $\sqrt{\text{MSE}}$ — same units as target.
- R^2 (Coefficient of Determination): $1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$ — proportion of variance explained (1.0 = perfect).

9.5 Cross-Validation

☰ K-Fold Cross-Validation

Split data into k equally-sized folds. Train on $k - 1$ folds, validate on the remaining fold. Repeat k times, rotating the validation fold. Report the **average** performance across all k folds.

Fold 1:	Validation	Train	Train	Train	Train
Fold 2:	Train	Validation	Train	Train	Train
Fold 3:	Train	Train	Validation	Train	Train
Fold 4:	Train	Train	Train	Validation	Train
Fold 5:	Train	Train	Train	Train	Validation

10 Regularization Techniques

💡 Regularization

Techniques to prevent overfitting by adding constraints or penalties to the model, discouraging complexity and improving generalization to unseen data.

Technique	Description
L1 (Lasso)	Adds $\lambda \sum w_i $ to loss. Drives some weights to exactly zero (feature selection).
L2 (Ridge)	Adds $\lambda \sum w_i^2$ to loss. Shrinks weights toward zero but doesn't eliminate them.
Elastic Net	Combines L1 and L2: $\lambda_1 \sum w_i + \lambda_2 \sum w_i^2$.
Dropout	Randomly deactivates neurons during training (neural networks). Prevents co-adaptation.
Early Stopping	Stop training when validation error starts increasing.
Data Augmentation	Artificially expand training data (rotations, flips, crops for images).
Pruning	Remove branches from decision trees that provide little predictive power.
Batch Normalization	Normalize layer inputs during training, stabilizes and speeds up learning.

11 Practical Considerations

11.1 Handling Imbalanced Data

⚠️ Class Imbalance

When one class greatly outnumbers the other (e.g., 95% negative, 5% positive), models tend to predict the majority class and appear accurate while failing on the minority class.

Solutions:

- **Resampling:** Oversampling minority (SMOTE), undersampling majority.
- **Class weights:** Assign higher cost to misclassifying minority class.
- **Evaluation:** Use Precision, Recall, F1, AUC instead of Accuracy.
- **Anomaly detection:** Treat minority class as anomalies.

11.2 Hyperparameter Tuning

- **Grid Search:** Exhaustively try all combinations of specified hyperparameter values.
- **Random Search:** Randomly sample combinations—often more efficient than grid search.
- **Bayesian Optimization:** Builds a probabilistic model to guide search toward promising regions.
- **Automated ML (AutoML):** Tools that automate model selection and tuning (e.g., Auto-sklearn, TPOT, H2O).

11.3 Feature Importance and Interpretability

- **Feature importance:** Random Forest and Gradient Boosting provide built-in scores.
- **SHAP (SHapley Additive exPlanations):** Model-agnostic explanations based on game theory.
- **LIME (Local Interpretable Model-agnostic Explanations):** Explains individual predictions.
- **Partial Dependence Plots:** Show the marginal effect of a feature on predictions.

12 Summary: Algorithm Selection Guide

Algorithm	Type	Best For	Scalability	Key Strength
Linear Regression	Regression	Linear relationships	High	Simple, interpretable
Logistic Regression	Classification	Binary classification	High	Probabilistic output
Decision Tree	Both	Interpretable models	Medium	Easy to visualize
Random Forest	Both	General-purpose	High	Robust, feature importance
XGBoost / LightGBM	Both	Tabular data	High	State-of-the-art accuracy
SVM	Both	Small-medium data	Low-Med	Effective in high dim
KNN	Both	Small datasets	Low	No training phase
Naïve Bayes	Classification	Text classification	High	Very fast, simple
K-Means	Clustering	Well-separated clusters	High	Simple, fast
PCA	Dim. Reduction	Preprocessing, viz	High	Captures max variance
Neural Network	Both	Complex patterns	Varies	Universal approximator
CNN	Classification	Images, spatial data	GPU needed	Spatial feature learning
Transformer	Sequence	NLP, generative AI	GPU needed	Attention mechanism

13 Best Practices Checklist

- ✓ Understand the problem type (classification, regression, clustering)
- ✓ Handle missing values and outliers
- ✓ Encode categorical variables properly
- ✓ Explore and visualize data before modeling (EDA)
- ✓ Scale/normalize features when required
- ✓ Split data into train/validation/test sets

- ✓ Start with simple baseline models first
- ✓ Use cross-validation for reliable estimates
- ✓ Tune hyperparameters systematically
- ✓ Monitor for overfitting (train vs. validation curves)
- ✓ Use appropriate metrics (not just accuracy)
- ✓ Address class imbalance if present
- ✓ Interpret and explain model decisions
- ✓ Document experiments and reproducibility

14 Common Pitfalls

⚠ Data Leakage

Information from outside the training dataset unintentionally leaks into the model (e.g., using test data for feature scaling, target leakage). Leads to overly optimistic performance that fails in production.

⚠ Not Enough Data

Complex models (deep learning) require large amounts of data. Using them on small datasets leads to severe overfitting. Start simple.

⚠ Ignoring Data Quality

“Garbage in, garbage out.” No algorithm can compensate for noisy, biased, or insufficient data. Invest in data cleaning and collection.

⚠ Wrong Metric

Using accuracy on imbalanced data, or MSE when outliers dominate. Always choose metrics aligned with the business objective.

⚠ Correlation vs. Causation

ML models find correlations, not causation. A model predicting outcomes does not mean the features *cause* the outcome.

15 ML Workflow — End-to-End Example

💡 Predicting House Prices — Workflow

1. **Define the problem:** Regression—predict continuous house price.
2. **Collect data:** Dataset with features (area, bedrooms, location, age, etc.) and target (price).
3. **EDA:** Visualize distributions, check correlations, identify missing values and outliers.
4. **Preprocess:** Handle missing values (median imputation), encode categorical features (one-hot for location), scale numerical features (StandardScaler).
5. **Split:** 80% train, 20% test; use 5-fold CV on training set.
6. **Train baseline:** Linear Regression → evaluate RMSE, R^2 .
7. **Try advanced models:** Random Forest, XGBoost → compare metrics.
8. **Tune:** RandomizedSearchCV on best model’s hyperparameters.

9. **Evaluate on test set:** Report final RMSE, MAE, R^2 .
10. **Interpret:** SHAP values to explain which features drive price predictions.
11. **Deploy:** Save model (pickle/joblib), build API, monitor performance.

16 Review Questions

1. Explain the difference between supervised, unsupervised, and reinforcement learning with examples.
2. What is the bias–variance tradeoff? How does model complexity relate to underfitting and overfitting?
3. Compare and contrast bagging and boosting. Name one algorithm for each.
4. What is the purpose of cross-validation? Explain k -fold cross-validation.
5. Define precision, recall, and F1-score. When would you prefer recall over precision?
6. What is regularization? Compare L1 (Lasso) and L2 (Ridge) regularization.
7. Explain how a decision tree makes predictions. What are Gini impurity and information gain?
8. What is the kernel trick in SVM? Why is it useful?
9. Describe the K-Means clustering algorithm and how to choose k .
10. What is data leakage and how can it be prevented?
11. Explain the architecture of a basic neural network (input, hidden, output layers, activation functions).
12. How would you handle a dataset with 95% negative and 5% positive examples?